

# Proactive Multi-Agent Orchestration for Intelligent Engagement on Education



## PROJECT REPORT

*Submitted by*

Group Number: 22

Sai Pranav Enjeti (1BM23CS286)

Prakrithi Jain (1BM23CS239)

Shashank Shantharam Nayak (1BM23CS313)

Siddharth Arya (1BM23CS328)

*in partial fulfillment for the award of the degree of*

**BACHELOR OF ENGINEERING**

in

**COMPUTER SCIENCE AND ENGINEERING**

Under the Guidance of

**Shreevatsa D S**

Assistant Professor, Department of CSE, BMSCE



**B.M.S. COLLEGE OF ENGINEERING**

(Autonomous Institution under VTU)

BENGALURU - 560019

Academic Year - 2025–2026

# Certificate

This is to certify that the project report entitled **“Proactive Multi-Agent Orchestration for Intelligent Engagement on Education”** is a bonafide work carried out by **Sai Pranav Enjeti (1BM23CS286), Prakrithi Jain (1BM23CS239), Shashank Shantharam Nayak (1BM23CS313), Siddharth Arya (1BM23CS328)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering**, under my supervision during the academic year 2025–2026.

Guide Signature  
(Shreevatsa D S)

HoD signature  
Dr. Kavitha Sooda

Principal Signature  
Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

# DECLARATION

We, hereby declare that the Major Project Phase-1 work entitled “Proactive Multi-Agent Orchestration for Intelligent Engagement on Education” is a bonafide work and has been carried out by us under the guidance of Shreevatsa D S, Assistant Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

| SL. NO. | Student Name              | USN        | Student's Signature |
|---------|---------------------------|------------|---------------------|
| 1       | Sai Pranav Enjeti         | 1BM23CS286 |                     |
| 2       | Prakrithi Jain            | 1BM23CS239 |                     |
| 3       | Shashank Shantharam Nayak | 1BM23CS313 |                     |
| 4       | Siddharth Arya            | 1BM23CS328 |                     |

Place: Bengaluru

Date:

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work of titled “Proactive Multi-Agent Orchestration for Intelligent Engagement on Education” as Major Project Phase-1 work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The work is original and I duly certify the same.

Guide Name:

Date:

Signature:

# **Acknowledgment**

We express our sincere gratitude to the Management of BMS College of Engineering and our Principal for providing a supportive environment and the necessary facilities to pursue our academic endeavors.

We are deeply indebted to Dr. Kavitha Sooda, Head of the Department, Computer Science and Engineering, for her constant encouragement and for permitting us to proceed with this project using the specialized computational resources housed within the department.

Our heartfelt thanks go to Dr. Nandhini Vineeth and the esteemed members of the review panel. Their technical suggestions, critical reviews, and expert guidance were instrumental in helping this project mature from a concept into its final form.

We would also like to thank our guide Shreevatsa D S for his valuable insights and technical contributions throughout the development process.

Finally, we are grateful to our families, friends, and all those who contributed, either directly or indirectly, to the successful completion of this project.

# Contents

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                             | <b>1</b>  |
| 1.1      | Overview . . . . .                                              | 1         |
| 1.2      | Motivation . . . . .                                            | 1         |
| 1.3      | Objectives . . . . .                                            | 1         |
| 1.4      | Scope . . . . .                                                 | 2         |
| 1.5      | SDG Justification . . . . .                                     | 2         |
| 1.6      | Existing System . . . . .                                       | 3         |
| 1.7      | Proposed System . . . . .                                       | 3         |
| 1.8      | Work Plan . . . . .                                             | 3         |
| <b>2</b> | <b>Literature Survey</b>                                        | <b>4</b>  |
| 2.1      | Overview . . . . .                                              | 4         |
| 2.2      | Detailed Survey . . . . .                                       | 5         |
| 2.3      | Research Gap Identification . . . . .                           | 6         |
| 2.4      | Mapping of Objective with Research Gap Identification . . . . . | 6         |
| <b>3</b> | <b>Software and Hardware Requirement Specification</b>          | <b>8</b>  |
| 3.1      | Functional Requirements . . . . .                               | 8         |
| 3.2      | Non-functional Requirements . . . . .                           | 8         |
| 3.3      | Hardware Requirements . . . . .                                 | 9         |
| 3.3.1    | For Training and Development . . . . .                          | 9         |
| 3.3.2    | For Production and Deployment . . . . .                         | 9         |
| 3.4      | Software Requirements . . . . .                                 | 9         |
| 3.4.1    | Development and Machine Learning . . . . .                      | 9         |
| 3.4.2    | System Design and Orchestration . . . . .                       | 10        |
| 3.5      | Cost Estimation . . . . .                                       | 10        |
| <b>4</b> | <b>Design</b>                                                   | <b>11</b> |
| 4.1      | High Level Design . . . . .                                     | 11        |
| 4.1.1    | System Components . . . . .                                     | 12        |
| 4.2      | Data Layer . . . . .                                            | 12        |
| 4.3      | System Architecture . . . . .                                   | 12        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 4.3.1    | Ingress and Data Handling . . . . .              | 12        |
| 4.3.2    | Stateful Agentic Orchestration . . . . .         | 13        |
| 4.3.3    | Micro-Agent Interactions . . . . .               | 13        |
| 4.3.4    | The Tri-Tier Data Persistence Strategy . . . . . | 13        |
| <b>5</b> | <b>Implementation</b>                            | <b>15</b> |
| 5.1      | Overview of Technologies Used . . . . .          | 15        |
| 5.2      | Implementation details of modules . . . . .      | 15        |
| 5.2.1    | AI Orchestrator . . . . .                        | 15        |
| 5.2.2    | User Profiling Module . . . . .                  | 15        |
| <b>6</b> | <b>Summary</b>                                   | <b>16</b> |
|          | <b>REFERENCES</b>                                | <b>17</b> |

# List of Figures

|     |                               |    |
|-----|-------------------------------|----|
| 4.1 | High Level Design . . . . .   | 11 |
| 4.2 | System Architecture . . . . . | 14 |

# List of Tables

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 2.1 | Mapping of Research Objectives to Identified Literature Gaps . . . . . | 7  |
| 4.1 | System Data Persistence Layers . . . . .                               | 14 |



# Abstract

With the fast growth of Artificial Intelligence (AI) and Large Language Models (LLMs), there is an increasing need for smart systems that go beyond simple reactions to engage users in a proactive and personalized way. Traditional AI systems mainly rely on single-model designs. They offer fixed responses with little adaptability and cannot remember user-specific context over time. These limitations reduce their effectiveness in situations that require ongoing interaction, personalization, and smart decision-making.

This project suggests creating a proactive multi-agent orchestration framework based on LLMs to support intelligent, context-aware user engagement. The system includes several specialized agents—such as conversational, analytical, recommendation, and evaluation agents—working together through an orchestration layer. This design allows the system to manage tasks dynamically, share responsibilities among agents, and create coherent and contextually relevant outputs. A key part of the system is persistent user profiling, which captures and updates user preferences, interaction history, and behavior patterns to support personalized responses. The proposed system can start interactions proactively, analyze user-generated content like documents and text, create meaningful outputs such as questions, summaries, topic mappings, and personalized feedback. It also supports ongoing learning by using past interactions to improve future recommendations and insights. The modular and scalable design ensures efficient deployment and real-time performance, making it suitable for real-world applications.

# **Chapter 1 Introduction**

## **1.1 Overview**

Modern education is shifting from passive learning to active, AI-enhanced discovery. This research introduces an Intelligent Tutoring System (ITS) based on a multi-agent LLM (Large Language Model) architecture. Instead of relying on a single chatbot, the system operates as a digital ecosystem of specialized agents, including Orchestration, Profiling, Content Analysis, and Question Generation. These agents work together to create a learning experience that mimics the responsiveness and adaptability of a human tutor.

## **1.2 Motivation**

The digital divide in education now involves more than internet access; it also includes access to quality, personalized instruction. Traditional e-learning platforms often serve as static repositories and do not solve the Two Sigma Problem, [1] which shows that students who receive one-on-one tutoring perform much better than those in traditional classrooms. Our motivation stems from the need to provide high-quality, personalized mentorship through an autonomous system that can recall a student's history, understand their uploaded materials, and step in when they face difficulties.

## **1.3 Objectives**

Our main goal is to create a recursive AI framework that goes beyond basic automation. Important milestones include:

- **Contextual Intelligence:** Building an LLM Orchestrator that manages state across sessions using short-term cache and long-term vector memory.
- **Autonomous Assessment:** Creating a pipeline that converts raw media (documents, images, videos) into structured knowledge and generates targeted evaluative questions.

- **Automated Feedback:** Creating feedback, reports and learning insights based on existing user persona and progress on a topic.
- **Behavioral Persistence:** Designing a profiling engine that captures and stores user preferences and knowledge gaps, ensuring each interaction builds on prior performance.

## **1.4 Scope**

This project focuses on designing and implementing a modular backend architecture. This involves developing an API Gateway for secure communication, a specialized data layer using Vector Databases for semantic search, and an “Agentic” layer where different LLMs perform specific tasks, such as the Insight Agent for reporting and the Proactive Agent for scheduling. The research emphasizes the technical integration of these components to create a seamless, real-time learning loop.

## **1.5 SDG Justification**

This project supports United Nations Sustainable Development Goal 4 (Quality Education). By automating time-consuming tutoring tasks, such as content summarization, assessment creation, and progress tracking, the system offers an affordable, impactful solution for learners worldwide. It fosters inclusive and equitable quality education by providing a personalized learning companion that is available around the clock, regardless of location or socio-economic status.

Beyond education (SDG 4), this project also aligns with Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure) and Sustainable Development Goal 10 (Reduced Inequalities).

From an industry perspective, the proposed multi-agent architecture contributes to the advancement of intelligent educational technologies by leveraging scalable LLM-based systems, vector databases, and autonomous agents. This reflects the broader shift toward AI-driven knowledge systems in the EdTech industry, where modular and interoperable AI components are becoming essential for building adaptive platforms.

In terms of inequality, traditional tutoring services are often expensive and geographically limited, creating barriers for learners in low-income or remote regions. By contrast, the proposed system aims to reduce educational inequality by providing personalized tutoring capabilities that are accessible anytime and anywhere. The system’s ability to persist user knowledge and adapt to individual learning needs helps bridge gaps in educational quality between different socio-economic groups.

## **1.6 Existing System**

Current Learning Management Systems (LMS) primarily serve administrative functions. They allow the delivery of PDFs and pre-recorded videos but do not address the learner's actual state of mind. These systems thus struggle to maintain a coherent conversation over prolonged study periods.

## **1.7 Proposed System**

We suggest a Multi-Agent Orchestration model. At its center, an LLM Orchestrator functions as the system's hub, directing user inputs to specialized agents based on intent.

- The Content Analysis Agent works on parsing documents to process study materials.
- The Profiling Agent updates a persistent User Profile Database, reflecting the learner's growth.
- The Question Generation Agent aligns content with the user's level of difficulty.

This modularity means the system is not just reactive but also predictive. An Insight Agent generates data-driven recommendations, while a Proactive Agent re-engages users through a scheduling feature.

## **1.8 Work Plan**

The project is divided into four phases:

1. Phase I (Conceptualization): Establishing inter-agent communication protocols and user data memory schema.
2. Phase II (Core Scaffolding): Creating the API Gateway and the central Orchestrator to handle request-response cycles and maintain state.
3. Phase III (Specialization): Developing and fine-tuning the specialized agents (Profiling, CAA, QGA) and integrating the Vector Database for semantic content retrieval.
4. Phase IV (Refinement): Implementing the Proactive Scheduler and conducting thorough testing to ensure the Response Composer provides effective and user-friendly feedback.

## **Chapter 2 Literature Survey**

The educational landscape has undergone a significant transformation over recent decades, transitioning from traditional physical instruction to the large-scale delivery of Massive Open Online Courses (MOOCs). Despite the increased accessibility of online learning platforms, providing instruction tailored to individual student needs remains a challenge—a manifestation widely characterized as the “Two Sigma Problem” [1]. Conventional digital platforms often rely on video instruction similar to a classroom, which lack the personalization required to emulate one-to-one tutoring.

However, the integration of Artificial Intelligence (AI) and Large Language Models (LLMs) has created new opportunities to approximate the benchmarks defined by Bloom [1]. While LLMs offer promising capabilities in simulating intelligent dialogue, current implementations exhibit notable limitations in adaptive engagement. This literature survey identifies and analyzes these research gaps, assessing the state of AI-driven educational frameworks.

### **2.1 Overview**

This literature survey explores the current horizon of artificial intelligence in education, tracing its journey from a static support tool to an autonomous pedagogical partner. Systematic reviews by Chen et al. [2], Elbasi et al. [3] and Agbewali-Koku et al. [4] provide a foundational taxonomy of machine learning applications, while Mallik and Gangopadhyay [5] categorizes these into proactive methods, such as content planning, and reactive methods, including performance assessment.

The field is currently moving away from simple content delivery toward intelligent interaction. We see a transition where platforms no longer just host media but actively monitor student behavior to intervene before a learner loses interest. This shift is particularly visible in technical domains like Digital Signal Processing (DSP), where pedagogy has evolved from mathematical theory to interactive, simulation-based, and now AI-enhanced learning [6], [7].

In this literature survey, we filtered research studies using subsets of the following keywords: artificial intelligence in education; machine learning; deep reinforcement learning; student engagement; personalized learning; digital signal processing;

education; pedagogy; teaching methods; Active learning (AL), edge AI, edge training, online distillation, real-time training; educational data mining; engaged learning; intelligent tutoring systems; multi-agent system; smart education; hybrid system; large language models

## **2.2 Detailed Survey**

The literature survey reveals four primary pillars of AI integration:

1. **Multi-Agent Systems and Pedagogical Autonomy:**

Traditional e-learning often lacks human-like intuition. Recent work has focused on Multi-Agent Systems (MAS) to provide dynamic adaptation. Zakaria et al. [8] and Vuković et al. [9] demonstrate how multi-tutoring agents can observe and support student activity independently. Further advancements using the Von Neumann framework proposed by Jiang et al. [10] suggest a shift toward fully autonomous environments where AI agents manage personalized learning pathways and manage the orchestration of complex educational tasks.

2. **Predictive Analytics and Affective Computing:**

The ability to foresee student struggle is a cornerstone of modern educational AI. Bagunaid et al. [11] utilizes deep reinforcement learning for early warning systems, while Rizwan et al. [12] identifies factors affecting student performance in MOOCs through systematic review. Beyond academic scores, Hasan et al. [13] and emphasizes the transition toward affective tutoring systems, which incorporate emotional intelligence and student engagement levels [14] into the learning loop. Regarding technical implementation, Mahafdah et al. [15] and Aderghal et al. [16] demonstrate that deep learning architectures, specifically CNNs and xLSTM for knowledge tracing, significantly outperform traditional statistical models in capturing sequential learning patterns.

3. **Large Language Models and Cognitive Diagnosis:**

LLMs have introduced advanced reasoning capabilities to education. Zhang et al. [17] introduces the SPP-L<sup>2</sup> framework for performance prediction on learner-sourced questions, while Chen et al. [18] explores LLM-enhanced cognitive diagnosis for deeper student understanding. Practical applications include specialized tutoring tools like PETRA [19], which provides recursive assistance for dynamic programming; proactive UI agents for visual analytics [20] and MAT, a multi-agent tutoring module [8]. These systems move beyond simple text generation to active, concept-centric evaluation and remedial recommendations [21].

#### 4. Immersive Learning and the Edu-Metaverse:

The spatial dimension of digital education is being redefined through virtual environments. Illi and Elhassouny [22] and Teichmann [23] provide frameworks for the Edu-Metaverse, designing immersive virtual learning based on real-world processes. These environments aim to increase engagement by providing high-fidelity simulations that are particularly effective in research-based learning models [24], [25].

## 2.3 Research Gap Identification

Despite these technological strides, several critical gaps remain:

- **Concept Drift and Memory Decay:** Hajmohammed et al. [26] highlights that LLMs suffer from concept drift as language and educational contexts evolve. Furthermore, most current systems lack a persistent, long-term memory of student progress across different learning sessions, leading to fragmented learning experiences.
- **Multimodal Knowledge Extraction:** While AI can process text, there is a lack of integrated pipelines that can autonomously convert diverse raw media, such as instructional videos and complex technical documents, into structured, evaluative knowledge concepts [6], [21].
- **Fragmented Feedback Mechanisms:** Current reactive systems often provide generic grading rather than personalized reports based on a specific user persona and historical progress [5], [13].
- **Scalability of Agentic Systems:** While deep reinforcement learning is powerful for multi-agent systems, Nguyen et al. [27] notes that training stable agents in dynamic educational settings remains a significant challenge, often requiring high computational overhead that is difficult to scale for individual users [7].

## 2.4 Mapping of Objective with Research Gap Identification

The primary objective of this work is to develop an intelligent, adaptive learning system that resolves the disconnect between high-level AI theory and practical implementation. By addressing the identified research gaps, this work focuses on:

The proposed solution thus aims to directly target the Two Sigma Problem by providing a scalable, persistent, and context-aware one-to-one tutoring experience.

Table 2.1: Mapping of Research Objectives to Identified Literature Gaps

| <b>Objective</b>                  | <b>Proposed Implementation</b>                                                    | <b>Addressed Research Gap</b>                                               | <b>Key References</b> |
|-----------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------|
| <b>1. Contextual Intelligence</b> | LLM Orchestrator with short-term cache and long-term vector memory.               | Mitigation of concept drift and temporal memory decay across sessions.      | [8], [26]             |
| <b>2. Autonomous Assessment</b>   | Multimodal pipeline converting raw media into structured knowledge and questions. | Manual content bottlenecks and non-adaptive evaluation methods.             | [17], [21]            |
| <b>3. Automated Feedback</b>      | Persona-based report generation and affective learning insights.                  | Disconnect between raw predictive analytics and student-centered support.   | [13], [25]            |
| <b>4. Behavioral Persistence</b>  | Profiling engine for tracking preferences and cumulative knowledge gaps.          | Lack of persistent, long-term behavioral tracking in e-learning frameworks. | [9], [16]             |



## Chapter 3    Software and Hardware Requirement Specification

The successful implementation of an AI-driven educational framework requires a robust technical foundation that balances high-performance training capabilities with lightweight, accessible production environments. This chapter details the technical specifications and resource estimations necessary to support the multi-agent architecture and Large Language Model (LLM) workflows.

### 3.1    Functional Requirements

The system must satisfy the following core functionalities to meet the objectives of SDG 4, 9, and 10:

- **Real-time Tutoring:** The system shall provide immediate, context-aware responses to student queries using LLM-based agents.
- **Adaptive Learning Pathways:** The multi-agent framework shall analyze student performance to dynamically adjust the difficulty and type of remedial content.
- **User Profiling:** The system shall maintain a persistent record of student interactions to track progress over multiple sessions.
- **Knowledge Retrieval:** The backend must support vector-based searches to retrieve relevant educational materials from a central database.

### 3.2    Non-functional Requirements

- **Scalability:** The software architecture should be capable of handling an increasing number of concurrent users without significant degradation in response time.
- **Latency:** To ensure an effective pedagogical experience, agent response times should ideally remain below 2 seconds.

- **Availability:** The production system must be accessible via a stable internet connection with minimal downtime.
- **Cost-Efficiency:** The system should leverage open-source libraries and free-tier cloud services to minimize the financial barrier for educational institutions.

### **3.3 Hardware Requirements**

The hardware is categorized into two phases: the resource-intensive training/development phase and the optimized production/inference phase.

#### **3.3.1 For Training and Development**

- **Processor:** Intel Core i5/i7 (8th Gen+) or AMD Ryzen 5/7 for efficient multi-threaded processing.
- **RAM:** Minimum 8GB required; 16GB highly recommended for handling large NLP models.
- **GPU:** NVIDIA GTX 1650+ with CUDA support for accelerated model training, or the use of Google Colab T4/L4 GPUs.
- **Storage:** Minimum 200GB free disk space for datasets and model weights.
- **Connectivity:** Stable high-speed internet for downloading pre-trained model weights.

#### **3.3.2 For Production and Deployment**

- **Internet:** Stable connection for uninterrupted API calls and user access.
- **Storage:** 10GB free disk space for lightweight containerized deployment.

### **3.4 Software Requirements**

The software stack is designed around Python's ecosystem for AI and modern microservices frameworks for the backend.

#### **3.4.1 Development and Machine Learning**

- **Language:** Python 3.12+ for Machine Learning; Java and Python for Backend services.

- **IDE:** VS Code, Jupyter Notebook, or Google Colab.
- **ML/DL Libraries:** TensorFlow, and Scikit-learn for model building and evaluation.
- **NLP Frameworks:** HuggingFace Transformers, spaCy, and NLTK for natural language understanding.
- **Data Handling:** Pandas and NumPy for preprocessing and matrix operations.

### **3.4.2 System Design and Orchestration**

- **Multi-agent Frameworks:** LangChain, CrewAI, or AutoGen for orchestrating complex agent interactions.
- **Database:** MongoDB (NoSQL) or PostgreSQL (Relational), with Vector DBs such as FAISS or Qdrant for RAG (Retrieval-Augmented Generation).
- **Backend:** FastAPI or Quarkus for high-performance API endpoints.
- **Deployment:** Docker for containerization and Hugging Face Spaces for hosting the live application.

## **3.5 Cost Estimation**

The financial strategy for this project focuses on accessibility and sustainability. By utilizing modern open-source ecosystems, the project maintains a low cost-entry:

1. **Compute:** Primary development utilizes institutional computing resources and the free-tier of Google Colab.
2. **Hosting:** Lightweight deployment is managed via Hugging Face Spaces (Free Tier).
3. **Software:** Exclusively utilizes open-source libraries (e.g., Python, FastAPI, LangChain), eliminating licensing fees.

**Total Estimated Cost:** Minimal to Zero. Expenses are limited to optional cloud scaling if user demand exceeds free-tier thresholds.

## Chapter 4 Design

### 4.1 High Level Design

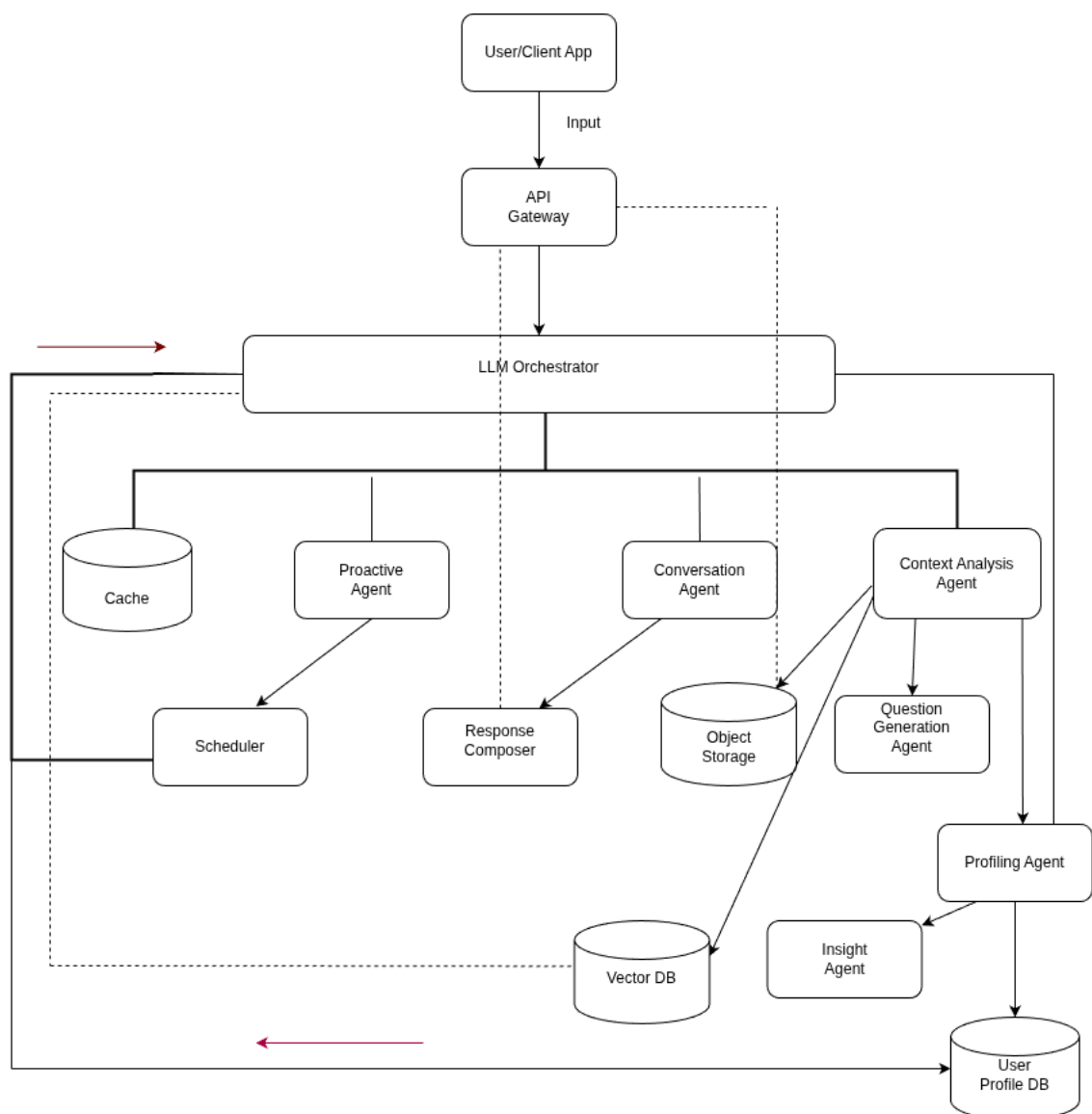


Figure 4.1: High Level Design

### 4.1.1 System Components

#### API Gateway

The entry point built with `FastAPI`, managing JWT authentication and routing file streams to **AWS S3/MinIO**.

#### LLM Orchestrator

The central coordinator using **LangGraph** to route requests between agents, referencing **Redis** for session state.

#### Specialized Agents

- **Profiling Agent (PA):** Syncs behavioral data to the **PostgreSQL** database.
- **Content Analysis Agent (CAA):** Processes raw files into searchable vectors.
- **Question Generation Agent (QGA):** Synthesizes assessments from indexed content.

## 4.2 Data Layer

The architecture utilizes a tri-tier storage strategy: **PostgreSQL** for relational user data, **Qdrant** for semantic vector search, and **Redis** for high-speed session caching.

## 4.3 System Architecture

The system architecture transitions from a linear request-response model to a state-aware, agentic ecosystem. This design ensures that the Large Language Model (LLM) is not merely a generator, but a central processor that interacts with specialized microservices and persistent data layers to deliver a personalized learning path.

### 4.3.1 Ingress and Data Handling

The entry point of the system is a high-performance **API Gateway** implemented via **FastAPI**. It manages asynchronous request handling and implements OAuth2-based JWT authentication. To ensure system scalability, the gateway utilizes a decoupled file handling strategy:

- **Object Storage Integration:** Large binary objects (PDFs, media files) are offloaded directly to **AWS S3** or **MinIO**.

- **Metadata Routing:** Only the metadata and object references are passed to the downstream agents, reducing the memory footprint of the orchestrator.

### 4.3.2 Stateful Agentic Orchestration

At the core of the intelligence layer lies the **LLM Orchestrator**, built using **LangGraph**. Unlike standard chain-based architectures, this graph-based approach allows for cyclic transitions and complex decision-making.

The orchestrator maintains a state object,  $S$ , defined as:

$$S = \{H_c, U_p, C_{ret}, T_{meta}\}$$

Where  $H_c$  represents the conversation history,  $U_p$  the user profile attributes,  $C_{ret}$  the context retrieved from vector storage, and  $T_{meta}$  the task-specific metadata. To maintain low-latency interactions, the **Short-Term Memory (STM)** is persisted in **Redis**, allowing the orchestrator to resume states across different API calls without re-processing the entire history.

### 4.3.3 Micro-Agent Interactions

The architecture employs a *Specialized Agent Cluster* where each node is a fine-tuned or prompt-engineered LLM instance:

- **Profiling Agent (PA):** Executes latent trait extraction. It monitors interactions for shifts in user proficiency and updates the **PostgreSQL** database via atomic UPSERT operations to ensure data integrity.
- **Content Analysis Agent (CAA):** Triggers a Retrieval-Augmented Generation (RAG) pipeline. It extracts text from S3 objects, performs semantic chunking, and generates high-dimensional embeddings for storage in **Qdrant**.
- **Question Generation Agent (QGA):** Synthesizes assessments by performing a semantic query on the **Vector DB** and cross-referencing the user's current knowledge level from the **User Profile DB**.

### 4.3.4 The Tri-Tier Data Persistence Strategy

The system architecture utilizes three distinct storage paradigms to optimize for consistency, speed, and semantic relevance:

This modular decoupling ensures that individual agents can be scaled or updated independently without disrupting the global orchestration logic, fulfilling the requirement for a resilient and adaptive learning environment.

| Layer            | Technology | Technical Utility                                     |
|------------------|------------|-------------------------------------------------------|
| Cache / STM      | Redis      | Session persistence and graph state management.       |
| Relational / LTM | PostgreSQL | ACID-compliant storage for user metrics and profiles. |
| Vector Knowledge | Qdrant     | HNSW-indexed semantic search for educational content. |

Table 4.1: System Data Persistence Layers

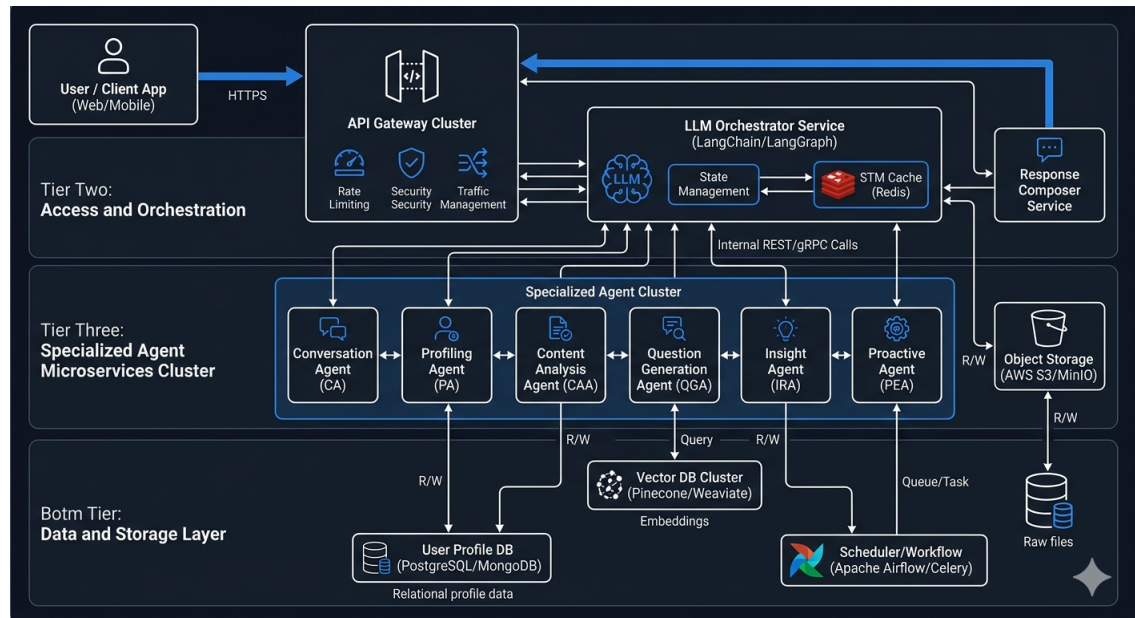


Figure 4.2: System Architecture

## Chapter 5 Implementation

### 5.1 Overview of Technologies Used

The proposed system shall be built on a modern AI stack: **FastAPI** for the backend, **LangGraph** for agent orchestration, **Qdrant** for vector storage, **PostgreSQL** for user metadata, and **Redis** for session caching.

### 5.2 Implementation details of modules

#### 5.2.1 AI Orchestrator

The AI Orchestrator is implemented as a state-driven workflow using **LangGraph**. It manages execution flow by maintaining a global `AgentState`, which encapsulates message history, user profiles, and task metadata.

- **State Management:** Uses `TypedDict` to ensure each agent has access to the necessary context (e.g., knowledge level, retrieved documents).
- **Conditional Routing:** Employs routing logic that dynamically directs the workflow based on the `current_task` (e.g., switching to `question_gen` for assessments).
- **Asynchronous Integration:** Interfaces with **Redis** for short-term session memory to ensure low-latency response times.

#### 5.2.2 User Profiling Module

Implemented via a dedicated `profiling_agent` that maintains a persistent understanding of the learner:

- **Dynamic Extraction:** Uses LLM function calling to analyze interactions and update the `knowledge_level` attribute.
- **Persistence Layer:** Updates the **PostgreSQL** `user_profiles` table using UPSERT operations to maintain a single source of truth.



## **Chapter 6   Summary**

The Two Sigma Problem is seen as the pinnacle of education, but as we've transitioned from traditional classrooms to modern LLMs, the experience is surprisingly fragmented. We have built tools that are powerful yet suffer from context drift, often forgetting a student's specific struggles the moment a session ends. This survey reveals that while Multi-Agent Systems and LLMs have opened new doors, the real challenge lies in bridging the gap between AI and the user. By addressing the "forgetfulness" of current models, we try to improve the interaction between user and the machine, helping the user understand concepts without losing valuable instruction.

## REFERENCES

- [1] B. S. Bloom, “The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring,” *Educational researcher*, vol. 13, no. 6, pp. 4–16, 1984.
- [2] L. Chen, P. Chen, and Z. Lin, “Artificial intelligence in education: A review,” *IEEE access*, vol. 8, pp. 75 264–75 278, 2020.
- [3] E. Elbasi, M. Nadeem, Y. I. Alzoubi, A. E. Topcu, and G. Varghese, “Machine learning in education: Innovations, impacts, and ethical considerations,” *IEEE Access*, 2025.
- [4] C. E. K. Agbewali-Koku, M. A. Rahman, M. Hamada, M. A. Ali, L. N. Oysharja, and M. T. Hossain, “A systematic review of machine learning techniques in online learning platforms,” in *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, IEEE, 2022, pp. 247–250.
- [5] S. Mallik and A. Gangopadhyay, “Proactive and reactive engagement of artificial intelligence methods for education: A review,” *Frontiers in artificial intelligence*, vol. 6, p. 1 151 391, 2023.
- [6] M. B. López, “Evolving pedagogy in digital signal processing education: Ai-assisted review and analysis,” *IEEE Access*, vol. 13, pp. 45 559–45 567, 2025.
- [7] M. Boldo, M. De Marchi, E. Martini, S. Aldegheri, and N. Bombieri, “Domain-adaptive online active learning for real-time intelligent video analytics on edge devices,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 4105–4116, 2024.
- [8] R. Zakaria, B. Hadhoum, E. Mourad, and M. Mustapha, “Dynamic adaptation in an intelligent multi-tutoring system: A multi-agent approach,” *IEEE Access*, 2025.
- [9] I. Vuković et al., “Multi-agent system observer: Intelligent support for engaged e-learning,” *Electronics*, vol. 10, no. 12, p. 1370, 2021.
- [10] Y.-H. Jiang et al., “Ai agent for education: Von neumann multi-agent system framework,” *arXiv preprint arXiv:2501.00083*, 2024.

- [11] W. Bagunaïd, N. Chilamkurti, A. S. Shahraki, and S. Bamashmos, "Visual data and pattern analysis for smart education: A robust drl-based early warning system for student performance prediction," *Future Internet*, vol. 16, no. 6, p. 206, 2024.
- [12] S. Rizwan, C. K. Nee, and S. Garfan, "Identifying the factors affecting student academic performance and engagement prediction in mooc using deep learning: A systematic literature review," *IEEE Access*, vol. 13, pp. 18 952–18 982, 2025.
- [13] M. A. Hasan, N. F. M. Noor, S. S. A. Rahman, and M. M. Rahman, "The transition from intelligent to affective tutoring system: A review and open issues.," *Ieee Access*, vol. 8, no. 8, pp. 204 612–204 638, 2020.
- [14] S. Ayouni, F. Hajjej, M. Maddeh, and S. Al-Otaibi, "A new ml-based approach to enhance student engagement in online environment," *Plos one*, vol. 16, no. 11, e0258788, 2021.
- [15] R. Mahafdah, S. Bouallegue, and R. Bouallegue, "Enhancing e-learning through ai: Advanced techniques for optimizing student performance," *PeerJ Computer Science*, vol. 10, e2576, 2024.
- [16] Y. Aderghal, H. Hafidi, and A. El Ghazi, "Xlstmkt: Xlstm for knowledge tracing," *IEEE Access*, 2025.
- [17] Z. Zhang, X. He, and Y. Zhang, "Spp-l<sup>2</sup>: A ppo-enhanced large language model framework for student performance prediction on learnersourced questions," *IEEE Access*, 2025.
- [18] X. Chen, J. Zhang, T. Zhou, and F. Zhang, "Llm-cdm: A large language model enhanced cognitive diagnosis for intelligent education," *Ieee Access*, 2025.
- [19] D. W. Hewedy and A. S. Abdelhady, "Petra: A personalized educational tutoring tool for recursive assistance leveraging multi-model llms for dynamic programming learning," in *2025 15th International Conference on Electrical Engineering (ICEENG)*, IEEE, 2025, pp. 1–6.
- [20] Y. Zhao, X. Shu, L. Fan, L. Gao, Y. Zhang, and S. Chen, "Proactiveva: Proactive visual analytics with llm-based ui agent," *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [21] N. Pradeesh, M. Thushara, K. A. Krishna, V. Pranav, and S. Krishnamoorthy, "Ai-driven personalized learning and remedial recommendation through knowledge concept-centric evaluation," *IEEE Access*, 2025.
- [22] C. Illi and A. Elhassouny, "Edu-metaverse: A comprehensive review of virtual learning environments," *IEEE Access*, 2025.

- [23] M. R. Teichmann, “How to design immersive virtual learning environments based on real-world processes for the edu-metaverse—a design process framework,” *IEEE Transactions on Learning Technologies*, vol. 18, pp. 100–118, 2025.
- [24] F. Festiyed, D. Desnita, Z. Natasya, M. A. Fadillah, and F. Novitra, “From assistance to autonomy: Ai integration in structured research-based learning for higher education,” *Electronic Journal of e-Learning*, vol. 24, no. 1, pp. 109–124, 2026.
- [25] A. Bakhouyi, A. Dehbi, L. Amhaimar, S. Broumi, M. Talea, and A. Khalidi, “Improving smart learning: Course completion via ai-driven hybrid system integration in big data,” *Telematics and Informatics Reports*, vol. 18, p. 100 199, 2025.
- [26] M. Hajmohammed, P. Chountas, and T. J. Chaussalet, “Concept drift in large language models: Challenges of evolving language, contexts, and the web,” in *2025 1st International Conference on Computational Intelligence Approaches and Applications (ICCIAA)*, IEEE, 2025, pp. 1–6.
- [27] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, “Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications,” *IEEE transactions on cybernetics*, vol. 50, no. 9, pp. 3826–3839, 2020.